



Département de génie informatique et génie logiciel

INF2010

Structures de données et algorithmes

Rapport TP1

Soumis par

Nicolas Mihali (2435049)

Akhan Mehmet Sozen (2420324)

Section 2

28 janvier 2026

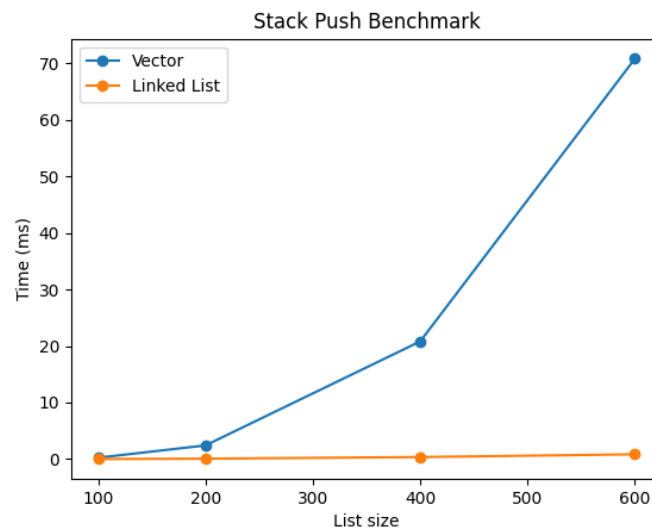
Polytechnique Montréal

Analyse de Stack Push

Le graphique de Stack Push (Figure 1), montre que le Vector est plus lent que la LinkedList, surtout lorsque la taille de la liste augmente, avec une courbe semblant quadratique ou bien exponentielle. En effet, cela est dû au fait que lorsque nous voulons insérer un élément dans un Vector ayant atteint sa taille maximale, il faut faire une copie des éléments et les mettre dans un autre Vector à taille plus grande, cela nécessite donc la création d'un nouveau tableau à chaque fois.

Il serait possible d'améliorer l'implémentation en stockant la taille de la liste dans une variable privée, car de nombreuses fonctions doivent appeler la fonction `size` dans Vector et celle-ci parcourt la liste entière afin d'obtenir la taille.

Figure 1 :

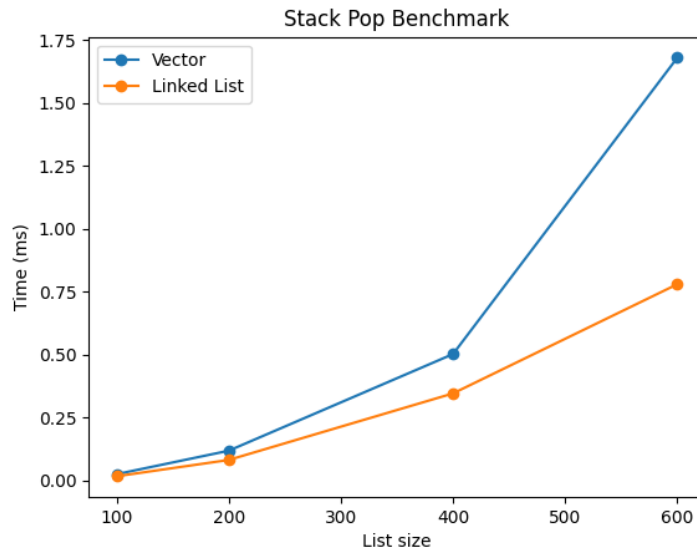


Analyse de Stack Pop

Le graphique de Stack Pop (Figure 2), montre aussi que le Vector est plus lent que la LinkedList. Utiliser `pop` sur Vector va appeler `remove` qui copie chaque élément dans un nouveau Vector en omettant le dernier élément. Pour LinkedList, on peut voir que `remove` devra absolument faire un parcours de la LinkedList.

Il serait possible d'améliorer l'implémentation de LinkedList en ajoutant un attribut « Node previous » dans la classe Node permettant ainsi d'avoir accès aux nœuds précédents. Aussi, avoir un attribut pour la taille.

Figure 2 :

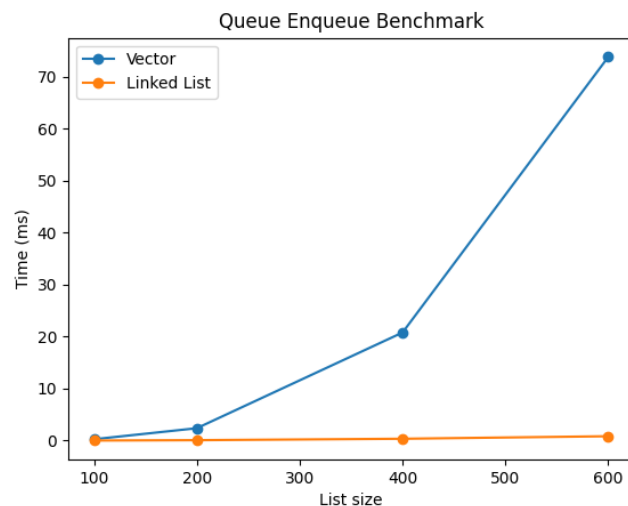


Analyse de Queue Enqueue

Le graphique de Queue Enqueue (Figure 2), montre une différence massive entre Vector et LinkedList. En effet, Vector est beaucoup moins performant, à partir de 400 éléments, un saut énorme se produit. La méthode `size()` appelée avec `insert()` rend chaque insertion de plus en plus demandant et en plus de cela, lorsque la capacité maximale est atteinte, un nouveau Vector doit être créé dans lequel est copié chaque élément.

Pour améliorer, un attribut `size` pourrait permettre d'éviter de rappeler `size()`.

Figure 3 :



Analyse de Queue Dequeue

Le graphique de Queue Dequeue (Figure 4), montre une situation similaire à Queue Enqueue. Cependant, Vector est presque deux fois moins performant comparé à la Figure 3. Cela s'explique par le fait que dans une LinkedList, dequeue appelle `remove(0)`, qui se charge simplement de déplacer le pointeur `head` à `head.next`, ce qui est une opération instantanée qui n'est pas influencée par la taille de la liste. Cependant, pour un Vector, il faut créer une nouvelle liste et parcourir la liste originale entière, ce qui prend plus de temps si la taille est grande.

Figure 4 :

